

Editorial

Autores: Gabriela Kwong Yamaguti, Guilherme Kiyotoshi Kavakami

Explicações dos Problemas

Fotos de Relíquias

Tags: Vetores

O problema nos pede para encontrar a quantidade de intervalos $[l, r]$ na paisagem que contêm exatamente uma única seção de destaque ($A_k = 1$).

Para resolver o problema de forma eficiente, podemos focar nas seções de destaque em si, em vez de iterar por todos os intervalos possíveis. Se fixarmos uma seção de destaque localizada no índice k , queremos saber quantos intervalos válidos têm k como a sua única seção de destaque. Para que isso aconteça:

- O limite esquerdo l do intervalo pode começar em qualquer posição desde a seção de destaque anterior (exclusive) até o próprio índice k .
- O limite direito r do intervalo pode terminar em qualquer posição desde o índice k até a próxima seção de destaque (exclusive).

Para fazer essa contagem de forma direta, armazenamos as posições de todas as seções de destaque em um vetor auxiliar idx . Para cada elemento $\text{idx}[i]$ (que representa a nossa i -ésima seção de destaque):

- **Cálculo das opções à esquerda:** Seja L o índice da seção de destaque anterior. Se for a primeira seção de destaque ($i = 0$), não há elemento anterior, então consideramos que a fronteira esquerda começa logo antes do primeiro elemento do vetor ($L = 0$). A quantidade de posições válidas para o início do intervalo é dada por $(\text{idx}[i] - L)$.
- **Cálculo das opções à direita:** Seja R o índice da próxima seção de destaque. Se for a última seção de destaque, não há elemento posterior, então consideramos a fronteira após o fim do vetor ($R = N + 1$). A quantidade de posições válidas para o fim do intervalo é dada por $(R - \text{idx}[i])$.

O número de intervalos válidos que contêm apenas a seção de destaque $\text{idx}[i]$ é o produto das opções de início e fim:

$$(\text{idx}[i] - L) \times (R - \text{idx}[i])$$

Somamos o resultado de cada seção de destaque na variável resp para obter a resposta final.

Complexidade: Iteramos uma única vez pelas seções de destaque inseridas em idx , realizando operações matemáticas em tempo constante $O(1)$ para cada uma. No pior caso, se todas as seções fossem de destaque, faríamos N passos. Portanto, a complexidade total do algoritmo é $O(N)$.

Escadaria

Tags: Vetores, Two Pointers

O problema nos pede para encontrar a maior altura possível para cada degrau de uma escadaria de modo que a diferença absoluta de altura entre degraus consecutivos seja no máximo 1. Como queremos maximizar as alturas, a estratégia ideal para preencher os degraus desconhecidos (valores -1) é fazê-los crescer o máximo possível, adicionando $+1$ a cada passo a partir das restrições dadas pelos degraus fixos conhecidos.

Podemos analisar o problema dividindo a escadaria em intervalos delimitados pelos índices que possuem alturas inicialmente conhecidas. Seja idx o vetor que armazena esses índices fixos. Para facilitar as condições de contorno (as pontas da escadaria), podemos adicionar dois degraus fictícios: um no índice 0 e outro no índice $N + 1$, ambos com valor inicial 0. Assim, todo degrau desconhecido estará estritamente entre dois degraus fixos L e R .

Para cada par de degraus conhecidos consecutivos $L = \text{idx}[i - 1]$ e $R = \text{idx}[i]$, queremos preencher o intervalo aberto (L, R) de forma a maximizar as alturas intermediárias. Existem três cenários possíveis para cada intervalo:

- **Intervalo na extremidade esquerda ($L = 0$):** Como o degrau 0 é fictício e não impõe uma restrição real vinda da esquerda, os degraus devem crescer o máximo possível da direita para a esquerda a partir de R . Portanto, iteramos de $R - 1$ até 1 fazendo $A_j = A_{j+1} + 1$.
- **Intervalo na extremidade direita ($R = N + 1$):** Analogamente, o degrau $N + 1$ é fictício. Os degraus devem crescer o máximo possível da esquerda para a direita a partir de L . Iteramos de $L + 1$ até N fazendo $A_j = A_{j-1} + 1$.
- **Intervalo interno ($L > 0$ e $R < N + 1$):** Quando ambos os limites são degraus reais fixos, a altura máxima permitida para os elementos internos é limitada pelas duas pontas simultaneamente. Vamos reduzir a distância entre as pontas usando uma abordagem de dois ponteiros:
 - Se $A_L > A_R$, o lado direito está muito baixo e pode crescer livremente em direção à esquerda. Subimos o degrau $R - 1$ definindo $A_{R-1} = A_R + 1$ e decrementamos R .
 - Se $A_L < A_R$, o lado esquerdo é quem está mais baixo e pode crescer em direção à direita. Subimos o degrau $L + 1$ definindo $A_{L+1} = A_L + 1$ e incrementamos L .

- Quando as alturas se igualam ($A_L = A_R$), ambos os lados passam a crescer simetricamente em direção ao centro do intervalo até que os dois ponteiros se encontrem.

Ao final do processamento de todos os intervalos, todas as posições originais com -1 terão sido substituídas pelo maior valor válido possível.

Complexidade: Durante o preenchimento dos intervalos, cada posição do vetor A é visitada e atualizada um número constante de vezes pelos ponteiros. Portanto, o tempo gasto para processar todos os intervalos de forma combinada é linear, resultando em uma complexidade total de $O(N)$.

Hidroviias e Rodovias

Tags: DSU

A condição para que uma rede de transportes seja simples (no máximo um caminho entre qualquer par de ilhas) significa que a rede não pode conter ciclos. Ou seja a rede deve ser um conjunto de árvores.

O problema nos diz que, há K anos, o sistema já continha todas as hidroviias atuais, mas continha no máximo K rodovias a menos (já que no máximo K rodovias foram construídas ao longo dos anos). Queremos saber se é possível que esse sistema antigo fosse simples.

Para maximizar a chance de o sistema antigo ser válido, devemos manter o maior número possível de rodovias atuais, desde que o sistema final continue sem ciclos. O processo pode ser modelado utilizando a estrutura de dados *Disjoint Set Union* (DSU):

- **Processamento das Hidroviias ($t = 1$):** As hidroviias já existiam no passado e não podem ser removidas. Portanto, inserimos todas as hidroviias primeiro no DSU. Se, ao tentar inserir uma hidrovia entre a e b , eles já estiverem no mesmo componente ($\text{find}(a) = \text{find}(b)$), significa que as próprias hidroviias já formam um ciclo. Como elas não podem ser removidas, a resposta será imediatamente inválida ($\text{resp} = \text{false}$).
- **Processamento das Rodovias ($t = 2$):** Armazenamos todas as rodovias para processá-las após as hidroviias. Para cada rodovia entre a e b :
 - Se a e b já estão no mesmo componente, esta rodovia fecharia um ciclo. Logo, ela **não poderia existir** no sistema do passado e deve ser contabilizada como uma das rodovias construídas nos últimos K anos ($\text{cont}++$).
 - Caso contrário, ela unirá dois componentes distintos sem formar ciclos, o que significa que ela poderia perfeitamente existir no passado sem violar a propriedade. Fazemos a união ($\text{une}(a, b)$).

Ao final do processo, se o número de rodovias que obrigatoriamente tiveram que ser removidas para evitar ciclos for menor ou igual a K ($\text{cont} \leq K$) e nenhuma inconsistência foi encontrada nas hidroviias ($\text{resp} = \text{true}$), então é possível que o sistema fosse simples há K anos.

Complexidade: Para cada uma das M conexões, realizamos operações de *find* e *une* no DSU. Utilizando a otimização de união por tamanho (*union by size*) e compressão de caminhos, cada operação toma tempo quase constante $O(\alpha(N))$. Portanto, a complexidade total do algoritmo é $O(M \cdot \alpha(N))$.

Harmonia Nasal

Tags: Guloso, Ordenação, Two Pointers, Implementação

- Parcial $K = N - 1$

Podemos pegar todos os elementos e usar as $N - 1$ deleções nos $N - 1$ primeiros elementos, e o b_N do último elemento nunca será usado (pois só subtraímos B quando adicionamos outro elemento). Portanto, a resposta será a soma de todos os a_i . Essa parcial nos dá uma observação importante: para facilitar a implementação, podemos pensar que K na verdade é $K + 1$, e sempre que escolhemos um elemento i somamos a_i e subtraímos b_i .

Complexidade: Basta somar todos os valores do vetor, então a complexidade é $O(N)$.

- Parcial $b_i = b_{i+1}$ para todo $1 \leq i \leq N$

Seja $B = b_1$. Note que para escolher em quais elementos vamos aplicar as deleções, apenas o valor a_i importa, já que todos os b_i são iguais. Assim, vamos começar pegando todos os elementos que possuem $a_i \geq B$. Se usarmos as deleções nos elementos que escolhemos, vamos ganhar B por deleção, e se usarmos nos elementos que sobraram, vamos ganhar a_i por deleção. Como os a_i dos elementos que sobraram são menores que B , então é ótimo escolher ganhar B .

Portanto, a resposta é a soma de todos os a_i tal que $a_i \geq B$, e depois tentamos somar B um número K vezes (o limite é o número de elementos que escolhemos), e depois somamos os outros a_i , em ordem não crescente.

Complexidade: É necessário ordenar os a_i que sobram, então a complexidade é $O(N \log N)$.

- Parcial $a_i \geq b_i$, para todo $1 \leq i \leq N$

Vale a pena escolher todos os elementos, pois sempre incrementamos a resposta em um valor maior ou igual a 0. Agora, vamos escolher K valores b_i para deixar de subtrair. Assim, basta escolher os maiores b_i .

Complexidade: Precisamos ordenar os b_i , então a complexidade é $O(N \log N)$.

- Parcial $K = 0$

Pelas parciais anteriores, vamos começar escolhendo todos os i tal que $a_i \geq b_i$. Agora, precisamos fazer uma escolha:

- podemos deletar um i que escolhemos, isto é, deixar de subtrair um b_i que escolhemos, e nesse caso é melhor escolher o maior b_i .
- podemos deletar um i que não escolhemos, isto é, somar um a_i que não escolhemos, e nesse caso é melhor escolher o maior a_i .

Portanto, vamos separar os i em dois grupos: no primeiro grupo colocamos os i tais que $a_i \geq b_i$, e no segundo colocamos os i tais que $a_i < b_i$. Vamos somar na resposta a_i e subtrair b_i de todos os i do primeiro grupo, e depois disso basta escolher o maior elemento: b_i do primeiro grupo ou a_i do segundo grupo.

Complexidade: É necessário ordenar cada um dos grupos, então a complexidade é $O(N \log N)$.

- Parcial $K \leq 10$

Vamos aplicar a ideia da parcial anterior, mas para escolher 10 elementos. Assim, podemos fixar um número $0 \leq c \leq 10$ de elementos que vamos pegar do primeiro grupo, e pegamos $10 - c$ do segundo grupo. Basta escolher a maior resposta.

Complexidade: Depois de ordenar os grupos, para cada c precisamos passar por $C - c$ valores, então a complexidade é $O(N \log N + C^2)$.

- Solução Completa

Vamos melhorar a ideia anterior: temos dois vetores ordenados, e queremos escolher um vetor ordenado. Ao invés de fazer esse procedimento de forma quadrática, podemos usar Two Pointers.

Complexidade: Depois de ordenar os grupos, precisamos usar um Two Pointers, que é linear, então a complexidade é $O(N \log N)$.

Energia

Tags: Matemática, Estruturas de Dados, Busca Binária, Small-to-Large, Implementação

- Parcial $Q = 0, N \leq 3000$

Basta iterar por todos os pares i, j tal que $1 \leq i < j \leq N$, e caso $E(i, j) = K$, somamos 1 na resposta.

Complexidade: Iteramos por cada par uma vez, então a complexidade fica $O(N^2)$.

- Parcial $N \leq 3000$

Vamos processar as perguntas do final para o começo. Após todas as perguntas, temos vários intervalos pequenos, e podemos calcular a resposta de cada intervalo do mesmo modo que a parcial anterior. Como estamos processando de do final para o começo, cada "desmoronamento" passa a ser uma "união" de intervalos.

Seja m o índice da união, L o intervalo da esquerda, R o intervalo da direita e $l \in L$ e $r \in R$ índices quaisquer. Em cada "união" podem surgir novos pares válidos, e temos que avaliar os seguintes casos:

- O elemento do meio cria novos pares válidos, isto é, para todo l verificamos se $E(l, m) = K$, e para todo r verificamos se $E(m, r) = K$.
- Algum elemento da esquerda forma um par válido com algum elemento da direita, isto é, para todo l e para todo r verificamos se $E(l, r) = K$.

Complexidade: Note que em cada união temos que considerar novos pares, e cada par só é considerado uma única vez. Assim, no final de todas as uniões, processamos cada par uma única vez. Assim, a complexidade fica $O(N^2)$.

Observação: Também é possível resolver usando Small-to-Large, tanto de forma online como offline. A ideia do Small-to-Large é processar as perguntas do final para

o começo, e para cada união de intervalos L e R , vamos iterar por cada elemento do menor intervalo. Fixado um elemento, iteramos pelos elementos do outro intervalo. O Small-to-Large passa por $O(N \log N)$ elementos, e cada elemento passa por $O(N)$ outros elementos para calcular a resposta. Assim, a complexidade fica $O(N^2 \log N)$. Nesse problema, podemos usar o mesmo argumento para calcular a resposta de forma online. Embora essa solução seja mais complexa e mais lenta, ela é útil para a solução completa.

- Parcial $Q = 0$.

Para cada j , $1 < j \leq N$, vamos encontrar de forma rápida quantos i existem tal que $1 \leq i < j$ e $E(i, j) = K$. Precisamos saber quantos i existem tal que:

$$\begin{aligned} E(i, j) = K &\iff |i - j| + |A_i - A_j| = K \iff \\ j - i + |A_i - A_j| &= K \iff \\ j - i + A_i - A_j = K, A_i &\geq A_j \text{ ou } j - i - A_i + A_j = K, A_i < A_j \iff \\ j - A_j - K = i - A_i, A_i &\geq A_j \text{ ou } j + A_j - K = i + A_i, A_i < A_j. \end{aligned}$$

Observe que as duas igualdades envolvem apenas os valores $i - A_i$ e $i + A_i$. Assim, vamos agrupar todos os índices de acordo com esses valores. Construiremos dois mapas:

- para cada valor de $i + A_i$, armazenamos um vetor com todos os índices que possuem esse valor;
- para cada valor de $i - A_i$, armazenamos outro vetor com todos os índices que possuem esse valor

Como percorremos os índices em ordem crescente, todos esses vetores ficam ordenados. Vamos pré-calcular esses mapas.

Agora, vamos calcular a resposta para todo j . Dado um j , queremos contar quantos índices anteriores satisfazem cada uma das duas equações.

Para a primeira equação, procuramos o vetor correspondente ao valor $j - A_j - K$. Todos os índices i presentes nesse vetor satisfazem $j - A_j - K = i - A_i$. Precisamos garantir que $A_i \geq A_j$, Assim:

$$j - A_j - K = i - A_i \iff j - i - K = A_j - A_i$$

Queremos que $A_i \geq A_j$, então:

$$j - i - K = A_j - A_i \leq 0 \iff i \geq j - K$$

Assim, basta contar quantos índices desse vetor pertencem ao intervalo. Para fazer isso, como o vetor é ordenado, basta usar uma busca binária no intervalo:

$$[\max(1, j - K), j - 1].$$

Fazemos o mesmo para a segunda equação: procuramos o vetor correspondente ao valor $j + A_j - K$, e precisamos garantir que $A_i < A_j$. Analogamente, temos o intervalo:

$$[\max(1, j - K + 1), j - 1].$$

Complexidade: Para cada j , primeiro encontramos os vetores correspondentes, que é $O(\log N)$ com map ou $O(1)$ com unordered map, e para cada vetor fazemos duas buscas binárias, que também é $O(\log N)$. Portanto, a complexidade total é $O(N \log N)$.

- Parcial $N \leq 60000$

É a mesma ideia que a solução completa. Dependendo da implementação, a constante pode ficar ruim e acabar excedendo o tempo limite, então passaria nessa parcial mas não ganharia os 100 pontos.

- Solução completa

Vamos recapitular o que já conseguimos: a segunda parcial nos mostrou um modo de lidar com as perguntas de forma eficiente, e a terceira parcial nos ensinou uma forma de calcular a resposta para um intervalo de forma rápida, de modo que podemos calcular a resposta de cada índice em $O(\log N)$. Vamos juntar essas duas ideias para a solução completa.

Vamos processar as perguntas online (também é possível fazer de forma offline, mas online é mais fácil de implementar), com uma ideia similar ao Small-to-Large. A ideia do Small-to-Large é passar apenas pelos elementos de um dos intervalos L, R , aquele que tiver menor tamanho.

Inicialmente, vamos calcular a resposta do vetor inicial com a ideia da parcial $Q = 0$. Depois, para cada pergunta que deleta m , vamos calcular quantos pares atravessam m , e subtrair esse valor da resposta.

Podemos usar a ideia da terceira parcial: para cada elemento do intervalo menor, vamos ver quantos pares ele pode formar. Por exemplo, suponha que R seja o intervalo menor. Queremos saber quais $j \in R \cup \{m\}$ conseguem fazer um par com algum elemento de L . Assim, seja l o elemento mais a esquerda de L , temos os intervalos:

$$[\max(l, j - K), m] \text{ e } [\max(l, j - K + 1), m]$$

Fazemos o mesmo caso L seja o intervalo menor, e obtemos os intervalos:

$$[m, \min(r, k + k - 1)] \text{ e } [m, \min(r, j + k)]$$

Assim, ao final de cada pergunta atualizamos a resposta.

Complexidade: O Small-to-Large avalia $O(N \log N)$ elementos, e o custo para avaliar cada elemento é $O(\log N)$ (encontrar um vetor e fazer uma busca binária). Portanto, a complexidade é $O(N(\log N)^2)$.

Implementações

Fotos de Relíquias

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  #define int long long
4  int32_t main (){
5      int n;
6      cin >> n;
7      vector<int> idx;
8      for (int i=1;i<=n;i++){
9          int a; cin >> a;
10         if (a) idx.push_back(i);
11     }
12     int resp=0;
13     for (int i=0;i<idx.size();i++){
14         int l=(i>0)? idx[i-1] : 0;
15         int r=(i<idx.size()-1) ? idx[i+1] : n+1;
16         resp+=(idx[i]-l)*(r-idx[i]);
17     }
18     cout << resp;
19
20 }
```

Escadaria

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  #define int long long
4  int32_t main (){
5      int n;
6      cin >> n;
7      vector<int> a(n+3), idx;
8      a[0]=0;
9      a[n+1]=0;
10     for (int i=1;i<=n;i++){
11         cin >> a[i];
12     }
13
14     for (int i=0;i<=n+1;i++) if (a[i]!=-1)idx.push_back(i);
15     for (int i=1;i<idx.size();i++){
16         int l=idx[i-1], r=idx[i];
17         if (l==0){
18             for (int j=r-1;j>0;j--) a[j]=a[j+1]+1;
19             continue;
20         }
```



```

21         if (r==n+1){
22             for (int j=l+1; j<r; j++) a[j]=a[j-1] +1;
23             continue;
24         }
25
26         while(r>l && a[l]>a[r]){
27             a[r-1]=a[r]+1;
28             r--;
29         }
30         while(r>l && a[l]<a[r]){
31             a[l+1]=a[l]+1;
32             l++;
33         }
34         while (r-l>1){
35             a[l+1]=a[l]+1;
36             a[r-1]=a[r]+1;
37             r--;
38             l++;
39         }
40
41     }
42     for (int i=1;i<=n;i++)cout << a[i] << " ";
43
44 }

```

Hidroviias e Rodovias

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  #define int long long
4  #define NMAX 100005
5  vector<int> pai(NMAX), tam(NMAX, 1);
6  int find(int v){
7      if (pai[v]==v)return v;
8      return pai[v]=find(pai[v]);
9  }
10 void une(int a, int b){
11     a=find(a);
12     b=find(b);
13     if (a==b) return;
14     if (tam[b]>tam[a])swap(a,b);
15     pai[b]=a;
16     tam[a]+=tam[b];
17 }
18
19
20 int32_t main (){
21     int n, m, k;

```

```

22     cin >> n>>m>>k;
23     int cont=0;
24     bool resp=true;
25     vector<pair<int,int>> aresta;
26     for (int i=0;i<=n;i++)pai[i]=i;
27     for (int i=0;i<m;i++){
28         int a, b, t;
29         cin >> a>>b>>t;
30
31         if (t==1){
32             if (find(a)==find(b)) resp=false;
33             une (a,b);
34         }
35         if (t==2){
36             aresta.push_back({a,b});
37         }
38     }
39     for (auto [a,b]:aresta){
40
41         if (find(a)==find(b))cont++;
42         else une(a,b);
43
44     }
45     if (cont<=k && resp) cout<< "S";
46     else cout << "N";
47
48 }

```

Harmonia Nasal

```

1  #include<bits/stdc++.h>
2  using namespace std;
3
4  #define int long long
5  #define pii pair<int, int>
6  #define pb push_back
7
8  int32_t main(){
9      ios_base::sync_with_stdio(false); cin.tie(NULL);
10
11     int n, k; cin >> n >> k;
12     ++k;
13
14     vector<int> a(n+10), b(n+10);
15     for(int i=1; i<=n; ++i) cin >> a[i];
16     for(int i=1; i<=n; ++i) cin >> b[i];
17
18     int resp = 0;

```

```

19     vector<int> v1, v2;
20     for(int i=1; i<=n; ++i){
21         if(a[i]>=b[i]) resp+=a[i], resp-=b[i], v1.pb(b[i]);
22         else v2.pb(a[i]);
23     }
24
25     sort(v1.rbegin(), v1.rend());
26     sort(v2.rbegin(), v2.rend());
27
28     int i=0, j=0;
29     while(true){
30         if(k==0) break;
31         if(i==v1.size() && j==v2.size()) break;
32         if(j==v2.size() || (i<v1.size() && v1[i]>=v2[j])) resp += v1[i], ++i;
33         else if(i==v1.size() || (j<v2.size() && v1[i]<=v2[j])) resp += v2[j], ++j;
34         --k;
35     }
36
37     cout << resp << "\n";
38 }

```

Energia

```

1  #include<bits/stdc++.h>
2  using namespace std;
3
4  #define int long long
5  #define pii pair<int, int>
6  #define pb push_back
7
8  // conta quantos valores existem entre l e r no vetor v
9  int query(vector<int> &v, int l, int r){
10     int ret = (--upper_bound(v.begin(), v.end(), r) - --lower_bound(v.begin(), v.end(), l));
11     return ret;
12 }
13
14 int32_t main(){
15     ios_base::sync_with_stdio(false); cin.tie(NULL);
16
17     int n, q, k; cin >> n >> q >> k;
18     vector<int> a(n+10);
19     for(int i=1; i<=n; ++i) cin >> a[i];
20
21     unordered_map<int, vector<int>> mp_soma, mp_sub;
22     for(int i=1; i<=n; ++i){
23         mp_soma[i+a[i]].pb(i);
24         mp_sub[i-a[i]].pb(i);
25     }

```

```

26
27     int resp = 0;
28     for(int j=1; j<=n; ++j){
29         int l1 = max(1LL, j-k), r1 = j-1;
30         if(l1<=r1) resp += query(mp_soma[j+a[j]-k], l1, r1);
31
32         int l2 = max(1LL, j-k+1), r2 = j-1;
33         if(l2<=r2) resp += query(mp_sub[j-a[j]-k], l2, r2);
34     }
35
36     cout << resp << "\n";
37
38     set<pii> s; //intervalos disjuntos atuais
39     s.insert({1, n});
40     while(q--){
41         int m; cin >> m;
42         auto it = --s.upper_bound({m, n+1});
43         int l = it->first, r = it->second, L = m-1, R = r-m, cur = 0;
44         if(L<=R){
45             for(int j=1; j<=m; ++j){
46                 int l1 = m, r1 = min(r, j+k-1);
47                 if(l1<=r1) cur += query(mp_soma[j+a[j]+k], l1, r1);
48
49                 int l2 = m, r2 = min(r, j+k);
50                 if(l2<=r2) cur += query(mp_sub[j-a[j]+k], l2, r2);
51             }
52         }else{
53             for(int j=m; j<=r; ++j){
54                 int l1 = max(l, j-k), r1 = m;
55                 if(l1<=r1) cur += query(mp_soma[j+a[j]-k], l1, r1);
56
57                 int l2 = max(l, j-k+1), r2 = m;
58                 if(l2<=r2) cur += query(mp_sub[j-a[j]-k], l2, r2);
59             }
60         }
61
62         resp -= cur;
63         cout << resp << "\n";
64
65         pii novo1 = {it->first, m-1}, novo2 = {m+1, it->second};
66         s.erase(it);
67         if(novo1.first<=novo1.second) s.insert(novo1);
68         if(novo2.first<=novo2.second) s.insert(novo2);
69     }
70 }

```
